CIS 465 Advance Web

Chapter 14 Ajax and Chapter 15 Node.js

In this assignment, you will build a web service using Node.js and Express that fetches and displays data from a real-world public API. Using the node_pokeman project as a good code example.

Modern web applications rarely work in isolation — they communicate with external services to retrieve and display live data. To handle this communication efficiently, we use asynchronous programming with async/await, which allows our server to request data without freezing or blocking other operations while it waits for a response.

Our Express server will act as a middleman (also called a proxy) between the browser and the external API, receiving requests from the frontend, fetching data from the API using node-fetch, and sending the results back to the browser as JSON.

On the frontend, a simple webpage built with HTML, CSS, and JavaScript will use the browser's built in fetch() function to request data from our Express server and dynamically display it on the page.

By the end of this project you will have hands on experience with the full request and response cycle — from the browser, through your own Node.js server, out to a third party API, and back again.

https://github.com/public-apis/public-apis

Top 20 Free APIs — No Key Required

🎮 **Fun & Games**

#	API	What it Returns	Example URL
1	**PokéAPI**	Pokémon data, stats, abilities	pokeapi.co/api/v2/pokemon/1
2	**Trivia DB**	Quiz questions by category	opentdb.com/api.php?amount=10
3	**Dad Jokes**	Random dad jokes	icanhazdadjoke.com
4	**Chuck Norris**	Random Chuck Norris jokes	api.chucknorris.io/jokes/random
5	**JokeAPI**	Jokes by category	v2.jokeapi.dev/joke/Any
6	**Harry Potter**	Characters, spells, houses	hp-api.com/api/characters
7	**D&D 5e**	Spells, monsters, classes	dnd5eapi.co/api/spells

---

🐾 Animals

#	API	What it Returns	Example URL
8	**Dog CEO**	Random dog images by breed	dog.ceo/api/breeds/image/random
9	**Cat Facts**	Random cat facts	catfact.ninja/fact
10	**RandomFox**	Random fox images	randomfox.ca/floof

🌍 World & Geography

#	API	What it Returns	Example URL
11	**REST Countries**	Country info, flags, population	restcountries.com/v3.1/all
12	**Universities**	World university list by country	universities.hipolabs.com/search?country=US

🚀 Space & Science

#	API	What it Returns	Example URL
13	**NASA APOD**	Astronomy photo of the day	api.nasa.gov/planetary/apod?api_key=DEMO_KEY
14	**Open Meteo**	Weather forecasts, temperature	api.open-meteo.com/v1/forecast?latitude=52&longitude=13&current_weather=true
15	**ISS Location**	Real time space station location	api.open-notify.org/iss-now.json

---

🧪 Testing & Fake Data

#	API	What it Returns	Example URL
16	**JSONPlaceholder**	Fake posts, users, todos, comments	jsonplaceholder.typicode.com/users
17	**Random User**	Fake user profiles with photos	randomuser.me/api
18	**Advice Slip**	Random pieces of advice	api.adviceslip.com/advice

---

📚 Books & Words

#	API	What it Returns	Example URL
19	**Open Library**	Books, authors, covers	openlibrary.org/api/books?bibkeys=ISBN:0451526538&format=json
20	**Dictionary API**	Word definitions, phonetics	api.dictionaryapi.dev/api/v2/entries/en/hello

Important steps:

(1) Create a project
    a. Add server.js to the **root** of your project folder
    b. Add index.html, app.js, and styles.css to a folder called **public**
(2) In a terminal window complete the following steps:
    a. Initialize node: npm init -y
    b. Install express: npm i express
    c. Start your express server: node server.js
    d. Open http://localhost:3000

Additional requirements:

1) Your goal is to NOT use AI for this assignment, but if you do, use it sparingly. If you use AI, you need to document where you used it in your code and justify your reason. I'm okay with you using it to get help, for very specific items, but not to write your code for you.

2) **MUST USE JSDoc:** Well-documented code, including a JavaScript program header that includes a summary, author's full name, date, and GitHub repository URL

3) Must use GitHub
    a. Invite me to your repository (dejohns2)
    b. Numerous commits which will prove who did the work
    c. If working in a group, balance work across code and README
    d. Professional README.md
        i. Fully explain what the program does
        ii. Console output
            -- or --
            Embedded screenshots or GIFs (stored in a subfolder)
            Bullet point list of what new concepts were used
        iii. links to the author's profiles

4) PDF peer-review or self-review in a subfolder. Use the other person's name as the file name. A peer review should focus on the person's strengths and possibly provide helpful suggestions for future group work.

5) Attach your Good, Bad, and Ugly (GBU) reflection written at a college level. IMPORTANT: Include your GitHub URL in this document.